



Universidad Autónoma de Baja California
Facultad de Ingeniería Mexicali
Ingeniería en Computación

Reporte de Práctica 08
Lenguaje de Alto Nivel y Bajo Nivel

Organización y Arquitectura de Computadoras
Ing. Félix Arredondo Santamaría

Moya Monreal Erick Anselmo — 1110604

Fecha de entrega
15/MAYO/2026

Mexicali, Baja California
29 de mayo de 2026

Índice

1. Introducción	2
2. Problemática y Justificación	2
3. Objetivos de la práctica	2
4. Marco Teórico	3
4.1. Lenguajes de Alto Nivel y de Bajo Nivel	3
4.2. Programación Híbrida e Interoperabilidad C/Ensamblador	4
4.3. La Estructura Libro y su Disposición en Memoria	4
4.4. Direccionamiento Indirecto y Recorrido de Arreglos	5
4.5. Comparación de Cadenas a Bajo Nivel	5
4.6. La Convención de Llamada <code>cdec1</code>	5
5. Procedimiento	6
6. Resultados con pruebas	7
6.1. Estructura de datos e interfaz en C	7
6.2. Función <code>agregar_libro</code>	8
6.3. Función <code>prestar_libro</code>	12
6.4. Función <code>devolver_libro</code>	14
6.5. Función <code>buscar_libro</code>	16
6.6. Función <code>listar_libros</code>	17
6.7. Resumen de las operaciones implementadas	19
7. Conclusiones	19
Referencias	20
A. Repositorio de código	20

1. Introducción

Cuando se desarrolla software, rara vez ocurre en un único nivel de abstracción o con un único lenguaje de programación. En la práctica profesional es frecuente combinar lenguajes de alto nivel, que ofrecen productividad y legibilidad, con lenguaje ensamblador, que otorga control absoluto sobre los registros, la memoria y el conjunto de instrucciones del procesador. Esta interoperabilidad permite delegar la lógica crítica a rutinas optimizadas a bajo nivel mientras la capa de presentación se mantiene sencilla y mantenible.

En esta práctica se documenta la construcción de un sistema de gestión para una biblioteca, en el cual la totalidad de la lógica (agregar, prestar, devolver, buscar y listar libros) se implementa en MASM x86 mediante bloques de ensamblador en línea `__asm`, mientras que el lenguaje C se limita estrictamente a la interfaz de usuario: el menú, la captura de datos por teclado y la impresión de resultados en consola. Se analiza el recorrido de un arreglo de estructuras en memoria, el cálculo manual de desplazamientos (*offsets*) sobre cada campo, la comparación de cadenas carácter por carácter y la invocación de funciones de C desde ensamblador respetando la convención de llamada `cdecl`.

2. Problemática y Justificación

A veces es necesario trabajar con sistemas computacionales que cuentan con poca memoria o requieren un rendimiento extremo, como en el caso de sistemas embebidos, controladores de dispositivos o software de tiempo real. En estos contextos, el lenguaje ensamblador se convierte en una herramienta indispensable para exprimir al máximo las capacidades del hardware, optimizar el uso de memoria y garantizar la eficiencia de cada instrucción ejecutada [1]. Sin embargo, la complejidad del ensamblador hacen que su uso exclusivo sea impráctico para proyectos grandes o con interfaces de usuario complejas.

Al trasladar la lógica de un sistema de gestión a lenguaje ensamblador se debe conocer el tamaño exacto en bytes de cada estructura, calcular a mano el desplazamiento de cada campo respecto a la dirección base, avanzar el puntero de un registro al siguiente sumando el tamaño completo del registro y recorrer las cadenas byte por byte hasta el carácter nulo.

3. Objetivos de la práctica

Objetivo General

Diseñar e implementar un sistema de gestión de biblioteca híbrido en el que la lógica completa resida en lenguaje ensamblador MASM x86 y la interfaz de usuario en lenguaje C, demostrando la interoperabilidad entre un lenguaje de alto nivel y uno de bajo nivel mediante el manejo explícito de estructuras, cadenas y la convención de llamada `cdecl`.

Objetivos Específicos

- **Delimitar y separar** las responsabilidades de cada lenguaje, confinando la totalidad de la lógica (búsqueda, validación, conteo y modificación de datos) a bloques de ensamblador en línea y reservando para C únicamente la captura de entrada y la impresión de salida.
- **Calcular e interpretar** los desplazamientos en bytes de cada campo de la estructura **Libro**, así como el tamaño total del registro, para recorrer correctamente el arreglo de estructuras en memoria mediante direccionamiento indirecto.
- **Implementar** la comparación de cadenas carácter por carácter en ensamblador, deteniéndose en el carácter nulo terminador, como sustituto de la función `strcmp` del lenguaje de alto nivel.
- **Aplicar** la convención de llamada `cdecl` para invocar una función de C desde un bloque `__asm`, apilando los parámetros en orden inverso y liberando la pila tras el retorno.
- **Verificar** el correcto funcionamiento de las cinco operaciones del sistema (agregar, prestar, devolver, buscar y listar) mediante pruebas de ejecución que contemplen tanto los casos válidos como los casos límite (libro inexistente, sin ejemplares disponibles e inventario completo).

4. Marco Teórico

4.1. Lenguajes de Alto Nivel y de Bajo Nivel

Los lenguajes de programación se clasifican habitualmente según su grado de abstracción respecto al hardware. Un lenguaje de **alto nivel**, como C, Java o Python, ofrece construcciones cercanas al razonamiento humano (variables tipadas, estructuras de datos, funciones) y delega en el compilador la traducción a instrucciones de máquina, ocultando los detalles de registros y direcciones de memoria [2]. Un lenguaje de **bajo nivel**, como el ensamblador, mantiene una correspondencia prácticamente uno a uno con el lenguaje máquina de una arquitectura específica, exponiendo de forma explícita los registros del procesador, los modos de direccionamiento y la organización de la memoria [3]. El lenguaje C ocupa una posición intermedia particular: aunque se considera de alto nivel, su modelo de memoria y su acceso a punteros lo hacen idóneo para coexistir con ensamblador dentro de un mismo programa.

4.2. Programación Híbrida e Interoperabilidad C/Ensamblador

La programación híbrida consiste en combinar dos o más lenguajes dentro de un mismo ejecutable, asignando a cada uno la tarea para la que resulta más adecuado. En el entorno de Microsoft Visual C++ (MSVC) para arquitecturas de 32 bits, esta combinación se logra mediante el bloque de *ensamblador en línea* (*inline assembly*), delimitado por la palabra clave `__asm { ... }` [4]. Dentro de este bloque, el código ensamblador puede acceder directamente, por su nombre, a las variables locales y globales declaradas en C, así como a los parámetros de la función anfitriona, ya que el compilador resuelve esos identificadores como referencias a memoria o a la pila. Esta característica elimina la necesidad de definir interfaces explícitas y convierte al ensamblador en línea en el mecanismo de interoperabilidad más directo y ergonómico para proyectos x86.

En este proyecto la división de responsabilidades es, el lenguaje C actúa exclusivamente como capa de presentación e interfaz (despliegue del menú, captura mediante `scanf_s` y `gets_s`, e impresión con `printf`), mientras que cada decisión, comparación, conteo y modificación de datos se ejecuta dentro de los bloques `__asm`.

4.3. La Estructura Libro y su Disposición en Memoria

El núcleo del sistema es un arreglo de estructuras de tipo **Libro**. A diferencia del lenguaje de alto nivel, donde se accede a un campo por su nombre, en ensamblador es indispensable conocer el **desplazamiento** (*offset*) en bytes de cada campo respecto a la dirección base de la estructura, así como el **tamaño total** del registro para poder avanzar al siguiente elemento del arreglo [1]. La estructura empleada y su disposición se resumen en la Tabla 1.

Tabla 1: Disposición en memoria de la estructura **Libro** (tamaño total: 112 bytes).

Campo	Tipo	Tamaño (bytes)	Offset
titulo	char[50]	50	+0
autor	char[50]	50	+50
anio	int	4	+100
cantidad	int	4	+104
cantidad_total	int	4	+108

Estos valores son los que aparecen literalmente en el código ensamblador: la instrucción `add edx, 112` avanza el puntero de un libro al siguiente, mientras que `[edx+100]`, `[edx+104]` y `[edx+108]` acceden respectivamente al año, a los ejemplares disponibles y al inventario total. El campo `cantidad` representa las copias disponibles para préstamo y

`cantidad_total` el total adquirido por la biblioteca; la diferencia entre ambos corresponde a los libros actualmente prestados.

4.4. Direccionamiento Indirecto y Recorrido de Arreglos

Para recorrer el arreglo de estructuras se emplea el direccionamiento indirecto, en el cual un registro contiene la dirección de memoria del dato a manipular. La directiva `LEA` (*Load Effective Address*) carga la dirección base del arreglo `biblioteca` en un registro puntero, y a partir de ahí la combinación de un desplazamiento constante (`[edx+104]`) con el avance del puntero (`add edx, 112`) permite visitar secuencialmente cada registro [5]. El registro `EBX` actúa como índice del libro actual y `ECX` como total de libros, replicando el patrón de un ciclo `for` del lenguaje de alto nivel mediante `CMP` y saltos condicionales.

4.5. Comparación de Cadenas a Bajo Nivel

En el lenguaje de alto nivel la comparación de dos títulos se reduce a una llamada a `strcmp`. En ensamblador esta operación debe construirse de forma manual, se recorren ambas cadenas simultáneamente con dos punteros (`ESI` para el título buscado y `EDI` para el almacenado), comparando byte a byte los caracteres con `CMP`. Si dos caracteres difieren, las cadenas no coinciden; si son iguales y se alcanza el carácter nulo (0) que marca el fin de cadena, la coincidencia es completa [3]. La instrucción `TEST al, al` se utiliza para detectar ese terminador, pues activa la bandera de cero (`ZF`) únicamente cuando el registro vale cero.

4.6. La Convención de Llamada `cdecl`

Para invocar una función de C desde ensamblador es obligatorio respetar una *convención de llamada*, que define cómo se pasan los parámetros y quién libera la pila. El lenguaje C utiliza por defecto la convención `cdecl`, en la cual los argumentos se apilan mediante `PUSH` en **orden inverso** (de derecha a izquierda según la firma de la función), de modo que el primer parámetro quede en la cima de la pila, y es la **rutina llamante** quien debe restaurar el puntero de pila tras el retorno [1]. Por ello, después de un `CALL` a una función con cuatro argumentos de 4 bytes, el código ensamblador ejecuta `add esp, 16` para liberar los 16 bytes apilados. Este mecanismo es el que utiliza la operación de listado para delegar la impresión de cada libro a una función de C.

5. Procedimiento

El desarrollo de la práctica se organizó en torno a la construcción del sistema de gestión, partiendo de la definición de la estructura de datos y avanzando hacia la implementación de cada función con su lógica completamente alojada en ensamblador. Las actividades realizadas fueron las siguientes:

1. **Definición de la estructura de datos.** Se declaró en C la estructura `Libro` con los campos solicitados (título, autor, año y cantidad de ejemplares) más un campo adicional `cantidad_total` para distinguir el inventario completo de los ejemplares disponibles. Se reservó un arreglo global `biblioteca[100]` y un contador `totalLibros`.
2. **Construcción de la interfaz en C.** Se programó la función `main` con un menú cíclico que captura la opción del usuario y los datos de cada libro mediante `scanf_s` y `gets_s`. Se confinó a C exclusivamente la entrada y la salida.
3. **Implementación de `agregar_libro`.** Se desarrolló la lógica en ensamblador para buscar si el título ya existe; de ser así, se incrementa el inventario (reabastecimiento), y de lo contrario se copian título y autor carácter por carácter, se almacenan año y cantidad, y se incrementa el contador de libros.
4. **Implementación de `prestar_libro`.** Se codificó la búsqueda del libro por título y la validación de disponibilidad, solo si existen ejemplares disponibles se decrementa el campo `cantidad`.
5. **Implementación de `devolver_libro`.** Se programó la operación inversa, incrementando `cantidad` siempre que no se supere el inventario total `cantidad_total`, evitando devoluciones inválidas.
6. **Implementación de `buscar_libro`.** Se implementó la búsqueda que, al encontrar el libro, devuelve a C los punteros y valores de cada campo para su posterior impresión por la interfaz.
7. **Implementación de `listar_libros`.** Se recorrió el arreglo completo en ensamblador, invocando para cada elemento una función de impresión escrita en C mediante la convención de llamada `cdecl`.
8. **Pruebas de ejecución.** Se cargó un conjunto inicial de libros y se ejecutaron las cinco operaciones, verificando tanto los casos exitosos como los casos límite (libro inexistente, sin ejemplares e inventario lleno).

6. Resultados con pruebas

6.1. Estructura de datos e interfaz en C

La base del sistema es la estructura `Libro` y el arreglo global que la contiene, mostrados en el Código 1. Cabe destacar que el tamaño de esta estructura es de **112 bytes** ($50 + 50 + 4 + 4 + 4$), valor que se utiliza repetidamente en el código ensamblador para avanzar de un libro al siguiente dentro del arreglo. Como los campos enteros quedan en desplazamientos múltiplos de 4 (offsets 100, 104 y 108), no se introduce relleno (*padding*) adicional por alineación.

```
1 typedef struct {
2     char titulo[50];
3     char autor[50];
4     int anio;
5     int cantidad;           // ejemplares disponibles
6     int cantidad_total;    // inventario completo
7 } Libro;
8
9 Libro biblioteca[100];
10 int totalLibros = 0;
```

Listing 1: Definición de la estructura `Libro` y variables globales.

La interfaz de usuario reside por completo en la función `main` (Código 2). Su única responsabilidad es desplegar el menú, capturar la entrada del usuario y derivar cada opción a la función de lógica correspondiente. Ninguna decisión sobre los datos de la biblioteca se toma en esta capa.

```
1 int main() {
2     int opc;
3     char titulo[50], autor[50];
4     int anio, cant;
5
6     SetConsoleOutputCP(CP_UTF8);
7     SetConsoleCP(CP_UTF8);
8     inicializar_datos();
9
10    while (1) {
11        printf("\n1. Agregar Libro\n2. Prestar Libro\n3. Devolver
12               Libro\n"
13               "4. Buscar Libro\n5. Listar Libros\n6. Salir\n0pcion
14               : ");
15        scanf_s("%d", &opc);
16        getchar();
17        switch (opc) {
18            case 1:
19                printf("Titulo: "); gets_s(titulo, 50);
```

```
18     printf("Autor: ");  gets_s(autor, 50);
19     printf("Año: ");    scanf_s("%d", &anio);
20     printf("Cantidad: "); scanf_s("%d", &cant);
21     agregar_libro(titulo, autor, anio, cant);
22     break;
23     case 2: printf("Titulo a prestar: ");  gets_s(titulo, 50);
24             prestar_libro(titulo); break;
25     case 3: printf("Titulo a devolver: "); gets_s(titulo, 50);
26             devolver_libro(titulo); break;
27     case 4: printf("Titulo a buscar: ");    gets_s(titulo, 50);
28             buscar_libro(titulo); break;
29     case 5: listar_libros(); break;
30     case 6: return 0;
31 }
```

Listing 2: Menú principal y captura de datos (interfaz en C).

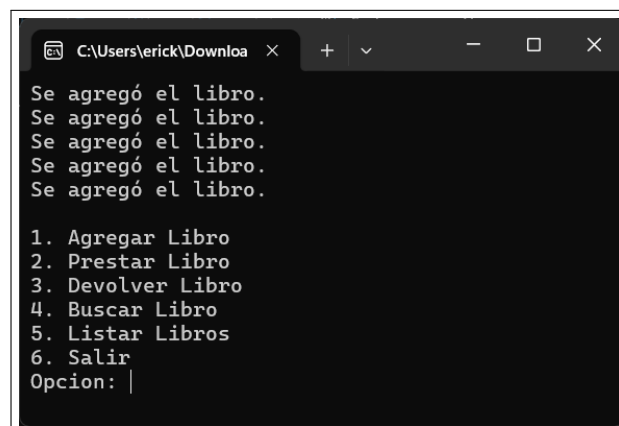


Figura 1: Menú principal del sistema de gestión de biblioteca desplegado por la interfaz en C.

6.2. Función agregar_libro

Esta función recibe desde C el título, el autor, el año y la cantidad de ejemplares. Primero se recorre el arreglo buscando si el título ya existe, si lo encuentra, suma la nueva cantidad tanto a los ejemplares disponibles como al inventario total (reabastecimiento); si no, copia los datos en el primer espacio libre y aumenta el contador de libros. El Código 3 muestra la implementación completa.

```
1  xor ebx, ebx           ; EBX = indice del libro actual = 0
2  mov ecx, totalLibros   ; ECX = total de libros existentes
3  lea edx, biblioteca    ; EDX = direccion base del arreglo
```

```
4
5 loop_verificar:
6     cmp ebx, ecx                ; recorrimos todos los libros?
7     jge es_nuevo               ; si -> el titulo no existe, es nuevo
8     push ecx                   ; preservar contador antes del compare
9     mov esi, t                 ; ESI -> titulo buscado
10    mov edi, edx                ; EDI -> titulo del libro actual
11 cmp_check:
12     mov al, [esi]
13     mov ah, [edi]
14     cmp al, ah                 ; comparar caracter a caracter
15     jne next_check
16     test al, al                ; llegamos al nulo terminador?
17     jz encontrado_update      ; si -> coincidencia completa
18     inc esi
19     inc edi
20     jmp cmp_check
21 next_check:
22     pop ecx
23     add edx, 112                ; avanzar al siguiente libro (+112 bytes
24     )
25     inc ebx
26     jmp loop_verificar
27 encontrado_update:
28     pop ecx
29     mov eax, cant
30     add [edx + 104], eax        ; sumar a ejemplares disponibles
31     add [edx + 108], eax        ; sumar al inventario total
32     mov resultado, 1           ; codigo: ya existia
33     jmp fin_agregar
34 es_nuevo:
35     mov eax, totalLibros
36     imul eax, eax, 112          ; offset del nuevo libro = total * 112
37     lea edi, biblioteca[eax]
38     mov esi, t                 ; copiar titulo
39     xor ecx, ecx
40 copy_titulo:
41     mov al, [esi + ecx]
42     mov [edi + ecx], al
43     test al, al
44     jz fin_titulo
45     inc ecx
46     jmp copy_titulo
47 fin_titulo:
48     add edi, 50                 ; avanzar al campo autor (offset +50)
49     mov esi, a
50     xor ecx, ecx
51 copy_autor:
52     mov al, [esi + ecx]
```

```

52     mov [edi + ecx], al
53     test al, al
54     jz fin_autor
55     inc ecx
56     jmp copy_autor
57 fin_autor:
58     add edi, 50                ; avanzar al campo anio (offset +100)
59     mov eax, an
60     mov [edi], eax
61     add edi, 4                ; cantidad disponible (offset +104)
62     mov eax, cant
63     mov [edi], eax
64     add edi, 4                ; cantidad_total (offset +108)
65     mov eax, cant
66     mov [edi], eax
67     inc totalLibros
68     mov resultado, 0         ; codigo: libro nuevo agregado
69 fin_agregar:

```

Listing 3: Lógica completa de `agregar_libro` en ensamblador en línea.

A continuación se analizan las instrucciones más representativas de esta rutina.

Análisis de instrucción

Instrucción

```
1     lea edx, biblioteca
```

Descripción Carga en EDX la dirección base del arreglo biblioteca, no su contenido.

A diferencia de MOV, que copiaría el *valor* almacenado en una dirección, LEA (*Load Effective Address*) calcula y carga la *dirección* efectiva del operando. Es el equivalente a obtener un puntero al primer elemento del arreglo, sobre el cual se aplicará posteriormente el direccionamiento indirecto para recorrer cada libro.

Análisis de instrucción

Instrucción

```
1     test al, al
2     jz encontrado_update
```

Descripción Detecta el carácter nulo terminador de la cadena: si AL vale cero, ambos títulos coincidieron por completo.

La instrucción TEST realiza un AND lógico entre sus operandos sin guardar el resultado, actualizando únicamente las banderas. Al aplicar TEST `al, al`, la bandera de cero (ZF) se activa solo si AL es cero. Como este punto del código únicamente se alcanza cuando los dos caracteres comparados son iguales, un valor nulo implica que se llegó simultáneamente al fin de ambas cadenas: la coincidencia es total. Este es el mecanismo que sustituye a

`strcmp` del lenguaje de alto nivel.

Análisis de instrucción

Instrucción

```
1  imul eax, eax, 112
2  lea edi, biblioteca[eax]
```

Descripción Calcula el desplazamiento del primer espacio libre del arreglo multiplicando el número de libros por 112 bytes.

Para situar el nuevo libro al final del arreglo se multiplica el índice (`totalLibros`) por el tamaño de la estructura (112 bytes). El resultado es el desplazamiento en bytes desde la base; sumándolo a `biblioteca` mediante `LEA` se obtiene la dirección exacta donde comenzar a escribir el título. Este cálculo manual es precisamente lo que el compilador de un lenguaje de alto nivel realizaría de forma automática al indexar `biblioteca[totalLibros]`.

Análisis de instrucción

Instrucción

```
1  add [edx + 104], eax
2  add [edx + 108], eax
```

Descripción Reabastecimiento: suma la nueva cantidad a los ejemplares disponibles (offset +104) y al inventario total (offset +108).

Cuando el libro ya existía, no se duplica el registro: se incrementan ambos contadores con la cantidad recibida. El uso de los desplazamientos +104 y +108 sobre el puntero `EDX` accede directamente a los campos `cantidad` y `cantidad_total`, conforme a la disposición de la Tabla 1.

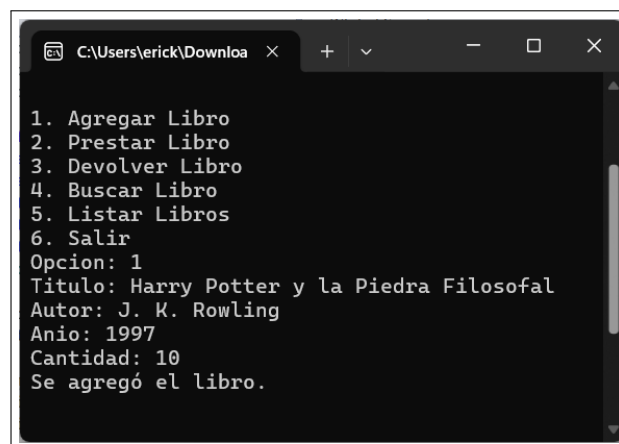


Figura 2: Resultado al agregar un libro nuevo a la biblioteca.

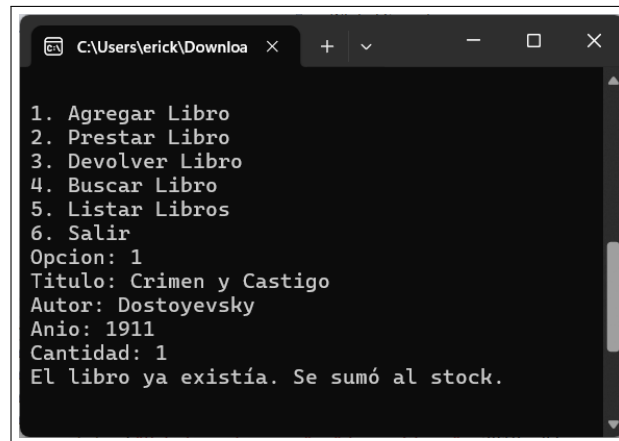


Figura 3: Reabastecimiento al agregar un título que ya existía en el inventario.

6.3. Función prestar_libro

Esta función localiza un libro por su título y, en caso de existir ejemplares disponibles, decrementa en uno el campo `cantidad`. Devuelve a C un código de estado: 1 si el préstamo fue exitoso, 2 si no quedan ejemplares y 0 si el libro no se encontró. El Código 4 presenta la rutina.

```
1 xor ebx, ebx
2 mov ecx, totalLibros
3 lea edx, biblioteca
4 loop_buscar:
5     cmp ebx, ecx
6     jge fin_búsqueda
7     push ecx
8     mov esi, t
9     mov edi, edx
10 cmp_str:
11     mov al, [esi]
12     mov ah, [edi]
13     cmp al, ah
14     jne next_book_pop
15     test al, al
16     jz found
17     inc esi
18     inc edi
19     jmp cmp_str
20 next_book_pop:
21     pop ecx
22     add edx, 112
23     inc ebx
24     jmp loop_buscar
25 found:
26     pop ecx
```

```
27     mov eax, [edx + 104]    ; ejemplares disponibles
28     cmp eax, 0
29     jle sin_stock          ; si <= 0, no hay disponibles
30     dec eax                ; un ejemplar menos
31     mov [edx + 104], eax
32     mov encontrado, 1      ; prestamo exitoso
33     jmp fin_busqueda
34 sin_stock:
35     mov encontrado, 2      ; sin ejemplares
36 fin_busqueda:
```

Listing 4: Lógica de prestar_libro: búsqueda y validación de disponibilidad.

Análisis de instrucción

Instrucción

```
1     mov eax, [edx + 104]
2     cmp eax, 0
3     jle sin_stock
```

Descripción Lee los ejemplares disponibles y verifica si quedan copias antes de prestar.

Esta es la validación central de la operación. Se carga en EAX el valor del campo `cantidad` (offset +104) y se compara contra cero. La instrucción JLE (*Jump if Less or Equal*) desvía el flujo a la etiqueta `sin_stock` cuando no hay ejemplares, impidiendo que el contador adopte valores negativos. Solo si la comparación indica que hay disponibilidad se ejecuta el DEC que reduce el inventario disponible. Toda esta decisión en C sería un `if (cantidad > 0)`.

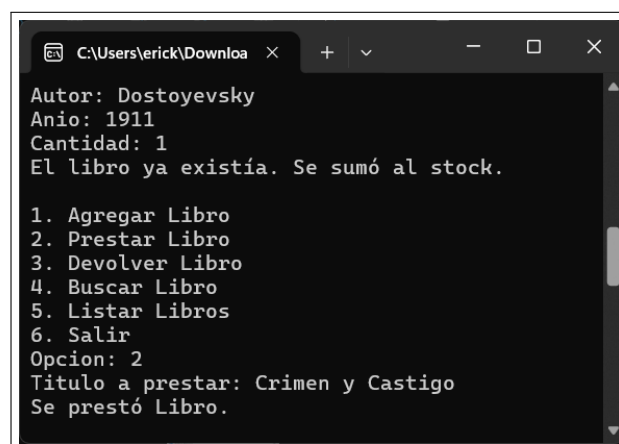


Figura 4: Préstamo exitoso de un libro con ejemplares disponibles.

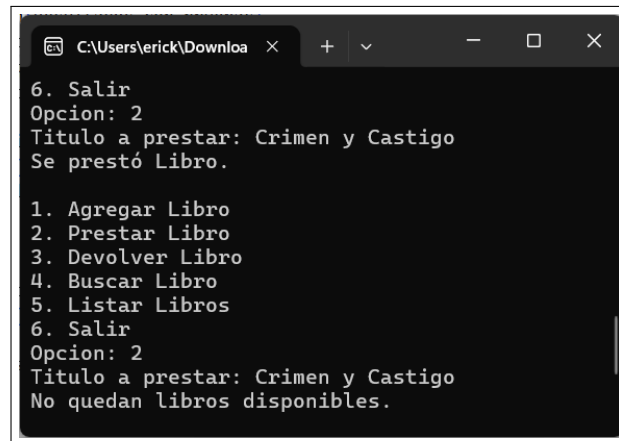


Figura 5: Intento de préstamo de un título sin ejemplares disponibles (caso límite).

6.4. Función devolver_libro

Es la operación inversa al préstamo, incrementa en uno el campo `cantidad`, pero solo si este no ha alcanzado el inventario total `cantidad_total`. De esta forma se evita registrar la devolución de más ejemplares de los que la biblioteca posee. El Código 5 muestra el bloque de validación, idéntico en su búsqueda a las rutinas anteriores.

```
1 found_dev :  
2     pop ecx  
3     mov eax, [edx + 104]    ; ejemplares disponibles  
4     mov ecx, [edx + 108]    ; inventario total  
5     cmp eax, ecx  
6     jge inventario_lleno    ; disponibles >= total -> nada que  
                             ; devolver  
7     inc eax                ; un ejemplar mas  
8     mov [edx + 104], eax  
9     mov encontrado, 1      ; devolucion exitosa  
10    jmp fin_búsqueda_dev  
11 inventario_lleno:  
12     mov encontrado, 2      ; ya estan todas las copias  
13 fin_búsqueda_dev:
```

Listing 5: Validación de `devolver_libro`: no exceder el inventario total.

Análisis de instrucción

Instrucción

```
1  mov eax, [edx + 104]
2  mov ecx, [edx + 108]
3  cmp eax, ecx
4  jge inventario_lleno
```

Descripción Compara los ejemplares disponibles contra el inventario total para impedir devoluciones inválidas.

Se cargan en sendos registros los campos `cantidad` (offset +104) e `cantidad_total` (offset +108). Si los ejemplares disponibles son mayores o iguales al total (JGE), significa que ninguna copia está prestada y, por tanto, no procede la devolución. Este control evita que el campo `cantidad` supere al inventario real, manteniendo la coherencia de los datos. La distinción entre ambos campos, disponibles frente a total, es justamente la razón por la que se añadió `cantidad_total` a la estructura original.

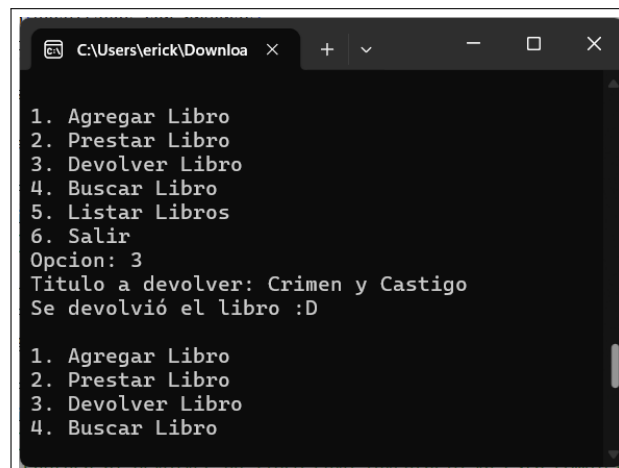


Figura 6: Devolución exitosa de un ejemplar previamente prestado.

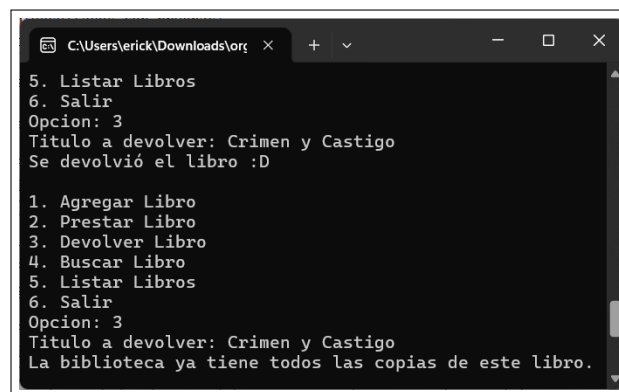


Figura 7: Intento de devolución con el inventario ya completo (caso límite).

6.5. Función buscar_libro

Esta función localiza un libro por su título y, al encontrarlo, entrega a C los datos de cada campo para que la interfaz los imprima. Aquí se aprecia con claridad la división de responsabilidades, el ensamblador resuelve la búsqueda y obtiene los punteros y valores, mientras que la impresión se delega a la función auxiliar `imprimir_libro_c`. El Código 6 muestra el bloque de extracción de datos.

```
1 found_info:
2     pop ecx
3     mov encontrado, 1
4     mov resTitulo, edx        ; puntero al titulo (offset +0)
5     lea eax, [edx + 50]
6     mov resAutor, eax        ; puntero al autor (offset +50)
7     mov eax, [edx + 100]
8     mov resAnio, eax         ; valor del anio (offset +100)
9     mov eax, [edx + 104]
10    mov resCant, eax         ; ejemplares disponibles (offset +104)
11 fin_búsqueda_info:
```

Listing 6: Extracción de los campos del libro encontrado en `buscar_libro`.

Análisis de instrucción

Instrucción

```
1     mov resTitulo, edx
2     lea eax, [edx + 50]
3     mov resAutor, eax
```

Descripción Devuelve a C las direcciones del título y del autor dentro de la estructura encontrada.

El título comienza en el offset +0, por lo que el propio puntero EDX ya es su dirección. Para el autor se emplea `LEA eax, [edx + 50]`, que calcula la dirección situada 50 bytes más adelante (el inicio del campo `autor`) sin leer su contenido. Estas direcciones se almacenan en variables de tipo `char*` de C, que luego se pasan a `printf`. El año y la cantidad, en cambio, se extraen por valor con `MOV`, ya que son enteros.

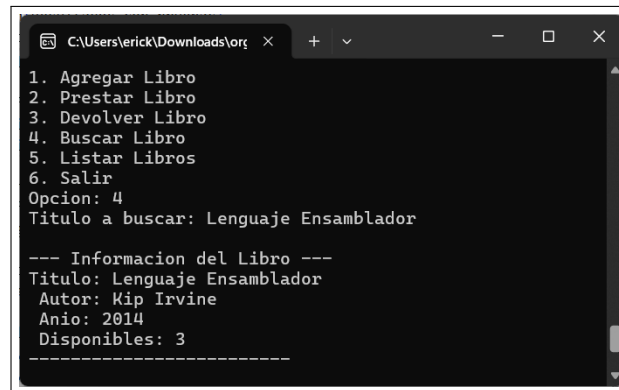


Figura 8: Resultado de una búsqueda exitosa con la información completa del libro.

6.6. Función listar_libros

La operación de listado recorre el arreglo completo y, para cada libro, invoca la función de impresión escrita en C. Este es el punto donde el ensamblador debe respetar la convención de llamada `cdecl` para llamar a una función del lenguaje de alto nivel. El Código 7 muestra la rutina.

```
1 xor ebx, ebx
2 lea esi, biblioteca
3 loop_listar:
4     cmp ebx, totalLibros
5     jge fin_listar
6     mov eax, [esi + 104]    ; 4to argumento: cantidad
7     push eax
8     mov eax, [esi + 100]    ; 3er argumento: anio
9     push eax
10    lea eax, [esi + 50]     ; 2do argumento: puntero a autor
11    push eax
12    push esi                ; 1er argumento: puntero a titulo
13    call imprimir_libro_c
14    add esp, 16             ; cdecl: el llamante libera 4*4 = 16
15                            bytes
16    add esi, 112            ; siguiente libro
17    inc ebx
18    jmp loop_listar
fin_listar:
```

Listing 7: Recorrido del arreglo e invocación de la función de C en `listar_libros`.

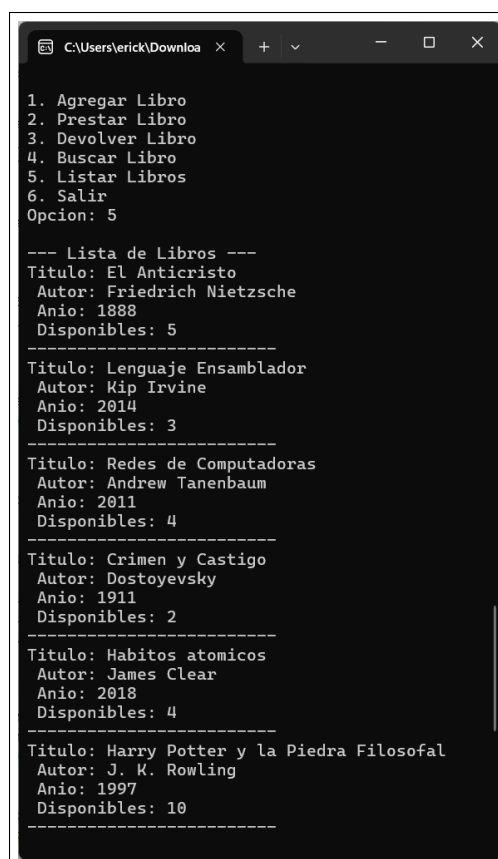
Análisis de instrucción

Instrucción

```
1  push eax      ; cantidad
2  push eax      ; anio
3  push eax      ; autor
4  push esi      ; titulo
5  call imprimir_libro_c
6  add esp, 16
```

Descripción Apila los cuatro argumentos en orden inverso, invoca la función de C y libera la pila según la convención `cdecl`.

La función de C tiene la firma `imprimir_libro_c(char* t, char* a, int an, int cant)`. Bajo la convención `cdecl`, los argumentos se apilan de derecha a izquierda: primero la cantidad, luego el año, después el puntero al autor y por último el puntero al título, de modo que este quede en la cima de la pila como primer parámetro. Tras el `CALL`, y dado que `cdecl` responsabiliza al **llamante** de limpiar la pila, se ejecuta `add esp, 16` para descartar los cuatro valores de 4 bytes apilados. Omitir esta corrección desbalancearía la pila y corrompería la ejecución en la siguiente iteración del ciclo.



```
C:\Users\erick\Downloads x + - □ x
1. Agregar Libro
2. Prestar Libro
3. Devolver Libro
4. Buscar Libro
5. Listar Libros
6. Salir
Opcion: 5

--- Lista de Libros ---
Titulo: El Anticristo
Autor: Friedrich Nietzsche
Anio: 1888
Disponibles: 5
-----
Titulo: Lenguaje Ensamblador
Autor: Kip Irvine
Anio: 2014
Disponibles: 3
-----
Titulo: Redes de Computadoras
Autor: Andrew Tanenbaum
Anio: 2011
Disponibles: 4
-----
Titulo: Crimen y Castigo
Autor: Dostoyevsky
Anio: 1911
Disponibles: 2
-----
Titulo: Habitos atomicos
Autor: James Clear
Anio: 2018
Disponibles: 4
-----
Titulo: Harry Potter y la Piedra Filosofal
Autor: J. K. Rowling
Anio: 1997
Disponibles: 10
-----
```

Figura 9: Listado completo de los libros registrados en la biblioteca.

6.7. Resumen de las operaciones implementadas

La Tabla 2 sintetiza la división de responsabilidades entre ambos lenguajes para cada operación del sistema, confirmando que la totalidad de la lógica reside en ensamblador y que C se limita a la entrada y salida.

Tabla 2: División de responsabilidades por operación.

Operación	Lógica en ensamblador	Rol de C
agregar_libro	Búsqueda de duplicados, copia de cadenas, escritura de campos, conteo	Captura de datos e impresión
prestar_libro	Búsqueda, validación de disponibilidad, decremento	Impresión del estado
devolver_libro	Búsqueda, validación contra el total, incremento	Impresión del estado
buscar_libro	Búsqueda y obtención de punteros/valores	Impresión de la ficha
listar_libros	Recorrido del arreglo y llamada <code>cdecl</code>	Impresión de cada libro

7. Conclusiones

Hacer esta práctica permitió comprender la cooperación entre un lenguaje de alto nivel y uno de bajo nivel dentro de un mismo programa. Al confinar la totalidad de la lógica al ensamblador y reservar para C únicamente la interfaz, se pudo observar todo aquello que un lenguaje de alto nivel resuelve de manera implícita, el cálculo de los desplazamientos de cada campo, el avance del puntero sumando el tamaño completo de la estructura y la comparación de cadenas carácter por carácter fueron tareas que tuvieron que programarse explícitamente, reforzando el tema de cómo se disponen realmente los datos en la memoria.

El manejo de la estructura `Libro` de 112 bytes resultó ilustrativo para manejar la memoria. Trabajar con desplazamientos fijos (+50, +100, +104, +108) sobre un puntero base hizo evidente que un acceso como `libro.cantidad` en C esconde, a nivel de máquina, una suma de dirección y un acceso indirecto a memoria.

El principal reto durante el desarrollo fue mantener la lógica en ensamblador y no filtrarla hacia C, resultaba tentador resolver una comparación o una validación con un simple `if` en el lenguaje de alto nivel, pero respetar la regla de la práctica obligó a expresar cada decisión mediante `CMP` y saltos condicionales.

Referencias

- [1] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th. Hoboken, NJ: Pearson Education, Inc., 2019, ISBN: 978-0-13-499719-3.
- [2] A. S. Tanenbaum y T. Austin, *Structured Computer Organization*, 6th. Pearson, 2013.
- [3] K. R. Irvine, *Assembly Language for x86 Processors*, 8th. Hoboken, NJ: Pearson, 2020, ISBN: 978-0-13-538165-6.
- [4] K. R. Irvine, *Assembly Language for x86 Processors*, 6th. Upper Saddle River, NJ: Prentice Hall, 2011.
- [5] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, ARM. Morgan Kaufmann, 2016.
- [6] A. Elahi, *Computer Systems: Digital Design, Fundamentals of Computer Architecture and ARM Assembly Language*, 2nd. Cham, Switzerland: Springer Nature Switzerland AG, 2022, ISBN: 978-3-030-93448-4. DOI: [10.1007/978-3-030-93449-1](https://doi.org/10.1007/978-3-030-93449-1).
- [7] J. L. Peterson, *Computer Organization and Assembly Language Programming*, Digital Edition. James L. Peterson, 2018.

A. Repositorio de código

El código completo del proyecto híbrido (C + **MASM x86**) desarrollado durante esta práctica se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/ApoloEM/OyAdeC/tree/main/practica8>